

Lynjax Beta 0.5

Intelligent Network Visibility

Manual de prueba con capturas virtuales

Backend FastAPI, Frontend React/Vite, lab local seguro y ruta de virtualización.

Fecha: 9 de junio de 2026

Criterio operativo acordado: los ambientes de prueba deben ejecutarse de forma virtualizada o sandboxed. No se debe tratar Windows local como runtime principal del lab. La virtualización por Intel Virtualization Technology fue confirmada desde BIOS, aunque algunos probes de sistema puedan reportar estados incompletos desde Git Bash/Windows.

Índice

1. Resumen ejecutivo	2
2. Política de virtualización y seguridad	2
3. Estructura del proyecto	2
4. Captura virtual del frontend	2
4.1. Funcionalidades visibles	2
5. Captura virtual del backend	4
5.1. Contratos iniciales del backend	4
6. Comandos de prueba	5
6.1. Validación local controlada	5
6.2. Arranque de demo frontend/backend	5
6.3. Virtualización beta	5
7. Criterios de aceptación	5
8. Próximos pasos	5

1. Resumen ejecutivo

Lynjax beta 0.5 es una base técnica para auditar, visualizar y documentar assessments de red con trazabilidad. Esta versión contiene:

- Backend FastAPI con endpoints de health, metadata y assessment simulado.
- Frontend React/Vite con dashboard visual de assessment, evidencia y reporte.
- Lab Docker Compose local para targets HTTP sanitizados.
- Manuales Markdown y esta versión LaTeX/PDF con capturas virtuales.
- Scripts de arranque, parada, smoke tests y probe read-only del host.

2. Política de virtualización y seguridad

Para evitar contaminar el host Windows y mantener pruebas reproducibles:

- Windows funciona como estación de control, no como runtime definitivo del lab.
- El runtime preferido para Docker/Compose es WSL2 Ubuntu/Debian o una VM Ubuntu con snapshot.
- No se deben tocar redes reales ni clientes sin autorización explícita.
- Los targets demo se limitan a loopback: 127.0.0.1.
- El endpoint actual de assessment es simulado y declara `network_access: disabled`.

3. Estructura del proyecto

backend/	API FastAPI, schemas, servicios simulados y tests de contrato.
frontend/	Dashboard React/Vite con identidad visual Lynjax.
lab/	Fixtures y targets locales seguros para pruebas repetibles.
virtualization/	Compose beta y wrapper para ejecutar app + lab en runtime Docker/Linux.
docs/	Manuales Markdown, estado de beta y páginas auxiliares de captura.
docs/manual/	Fuente LaTeX, PDF y capturas virtuales de frontend/backend.
reports/	Plantilla inicial de reporte técnico de assessment.
scripts/	Automatizaciones de dev, smoke, lab y probe del host.

4. Captura virtual del frontend

La Figura 1 muestra el dashboard visual de Lynjax. La captura fue generada desde una sesión de navegador automatizada sobre el build del frontend y almacenada dentro del proyecto en `docs/manual/assets/screenshots/f`

4.1. Funcionalidades visibles

- Navegación superior: Assessment, Evidencia y Reporte.
- Hero de producto con tagline *Intelligent Network Visibility*.
- Tarjetas de estado: activos visibles, hallazgos priorizados, evidencias y riesgo crítico.
- Panel de flujo controlado para checks de assessment.

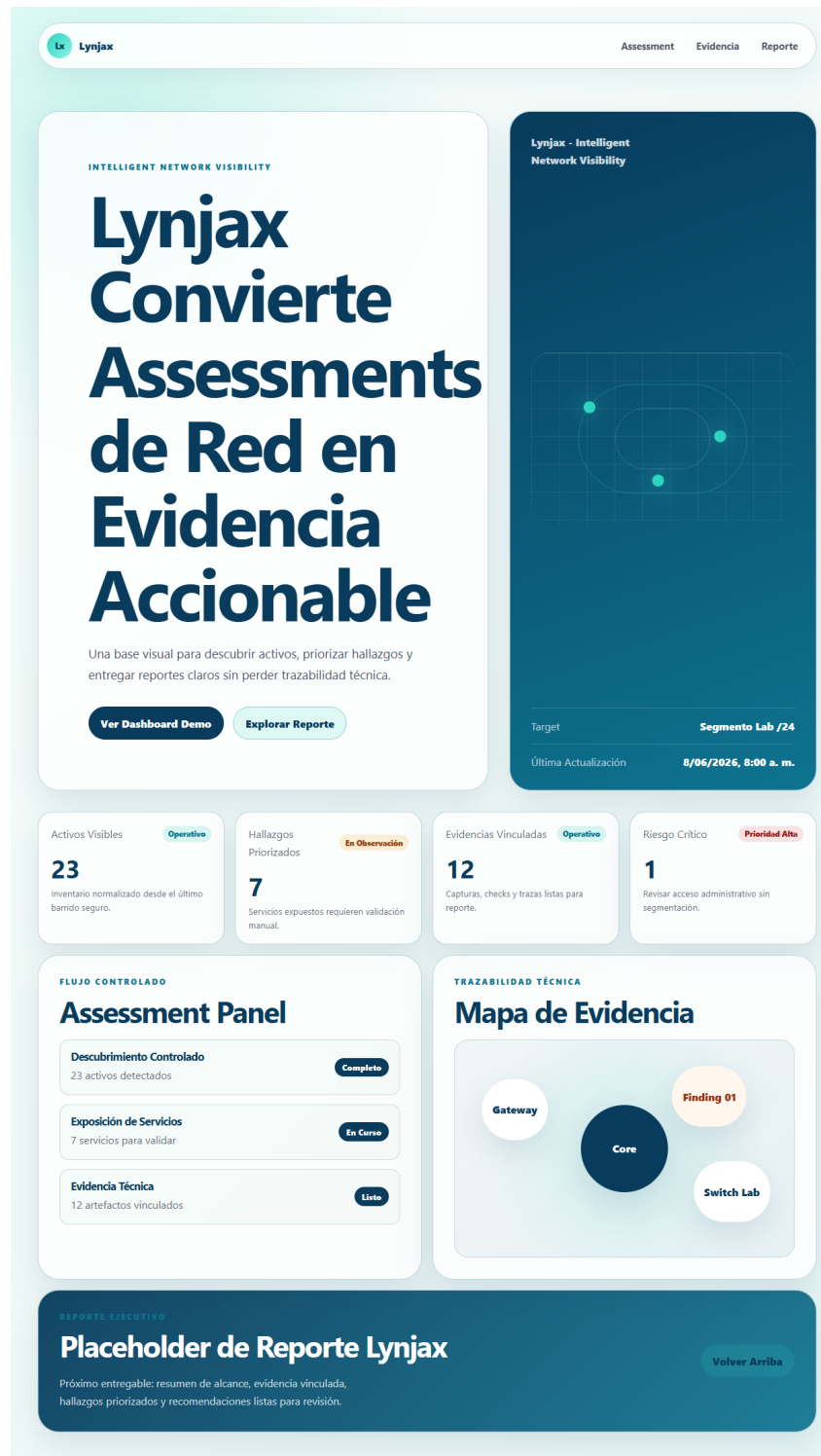


Figura 1: Dashboard frontend Lynjax con hero, métricas, panel de assessment, mapa de evidencia y placeholder de reporte.

- Mapa visual de evidencia con nodos de red y hallazgo.
- Sección de reporte ejecutivo como siguiente entregable.

5. Captura virtual del backend

La Figura 2 documenta la vista de backend preparada para manuales. Resume contratos API y una respuesta demo generada desde el backend FastAPI mediante cliente de pruebas, sin ejecutar probes contra redes reales.

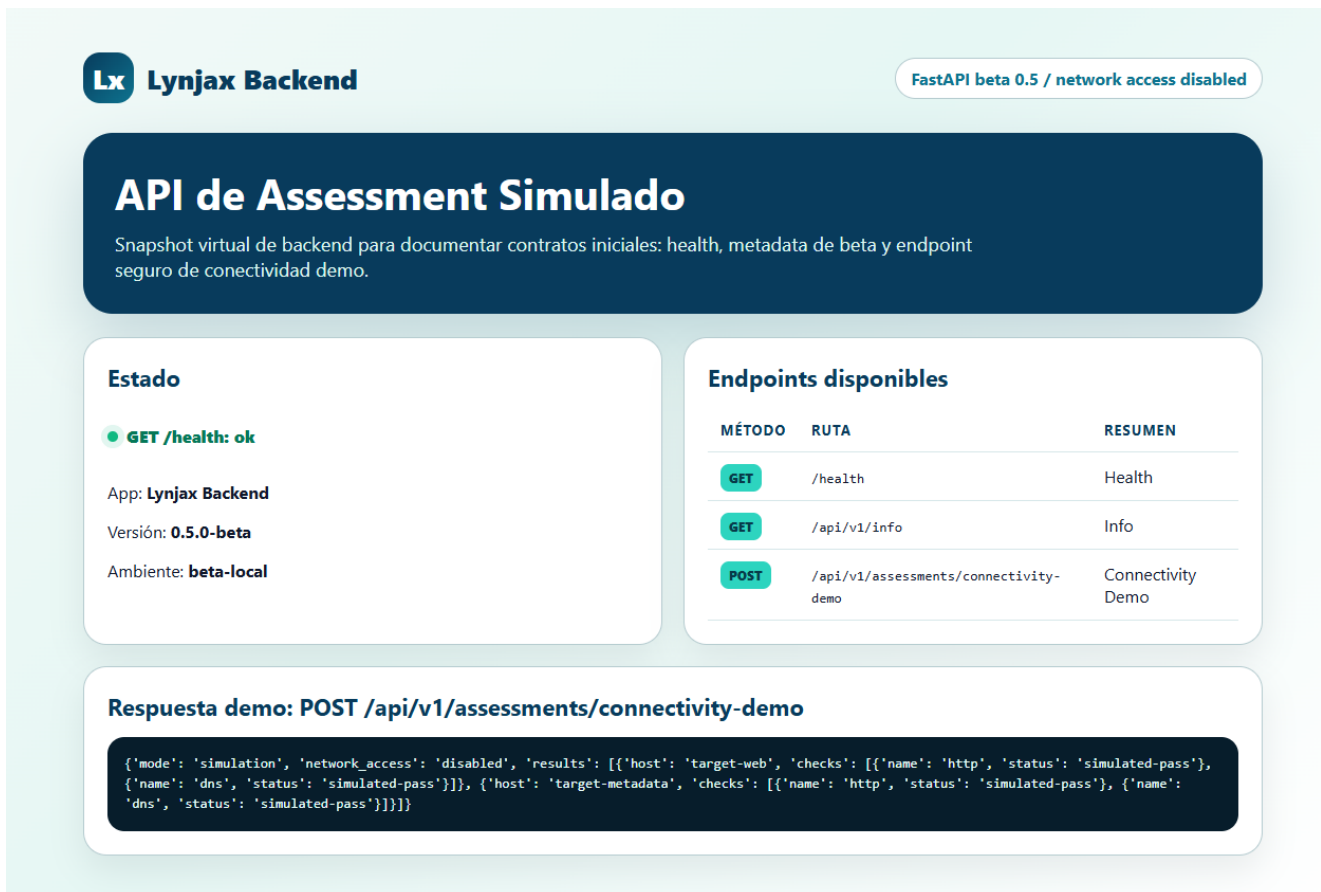


Figura 2: Snapshot virtual del backend Lynjux: health, endpoints disponibles y respuesta simulada de connectivity-demo.

5.1. Contratos iniciales del backend

- GET /health: verifica disponibilidad básica de la API.
- GET /api/v1/info: expone metadata de beta, versión, entorno y política de red.
- POST /api/v1/assessments/connectivity-demo: devuelve resultados simulados por host/check.

6. Comandos de prueba

6.1. Validación local controlada

Estos comandos validan artefactos del repo. Para el lab completo, ejecutar preferiblemente dentro de WSL2/VM/CI.

```
python -m pytest backend/tests -v
npm --prefix frontend run build
bash scripts/lab_validate.sh
```

6.2. Arranque de demo frontend/backend

```
bash scripts/dev-start.sh
bash scripts/smoke-local.sh
bash scripts/dev-stop.sh
```

6.3. Virtualización beta

```
bash scripts/host-probe.sh
cd virtualization
bash run-beta-compose.sh config
bash run-beta-compose.sh up
bash run-beta-compose.sh down
```

7. Criterios de aceptación

- Backend tests pasan: 3 passed.
- Frontend compila con Vite sin errores.
- Las capturas virtuales existen en docs/manual/assets/screenshots/.
- El PDF se genera en docs/manual/lynjax-beta-0.5-manual.pdf.
- El lab Compose se valida en WSL2/VM/CI cuando Docker esté disponible.

8. Próximos pasos

1. Conectar el frontend al endpoint real `/api/v1/assessments/connectivity-demo`.
2. Generar reportes Markdown/LaTeX/PDF desde datos devueltos por la API.
3. Añadir persistencia local mínima para ejecuciones y evidencias.
4. Mantener los manuales en tres formatos: Markdown, LaTeX y PDF.
5. Incluir capturas virtuales actualizadas en cada entrega funcional.